Real-Time System: Introduction

Module:5 Real Time Operating System :

Classification of Real time system, Issues & challenges in RTS, Real time scheduling schemes- EDF-RMS & Hybrid techniques, eCOS, POSIX, Protothreads.

A real-time system is a system that processes information and responds to events within specific time constraints. These systems are essential in applications where timing is critical, such as air traffic control, process control, and autonomous driving.

Here are some **characteristics of real-time systems:**

Respond to events : Real-time systems react to inputs from the environment and produce outputs that affect the environment.

Meet strict deadlines : Real-time systems need to meet strict deadlines to provide accurate and timely responses.

Highly reliable : Real-time systems are highly reliable and efficient for time-sensitive tasks.

Use a Real-Time Operating System (RTOS) : A key component of a real-time system is a Real-Time Operating System (RTOS). RTOSs process data and execute tasks within strict time constraints.

Some examples of real-time systems include:

Air traffic control systems
Process control systems
Autonomous driving systems
Anti-locking brakes
Airbags
Keyless entries
Climate control
GPS

Why we introduce the real-time system?

Introduction: Real-Time System

- Real-time
  - Does not denote speed/fast
  - But meet the timing constraints or deadlines
- Goal: make the system predictable and robust
- Predictability: have deterministic behavior
  - Usually use the worst-case analysis
  - Guaranteed response/reaction times
- Robustness: the job with granted performance will not be interfered with other job

Characteristics of Real-Time Systems

- Characteristics
  - Must produce correct computational results, called *logical or functional correctness*
  - These computations must conclude within a predefined period, called *timing correctness*

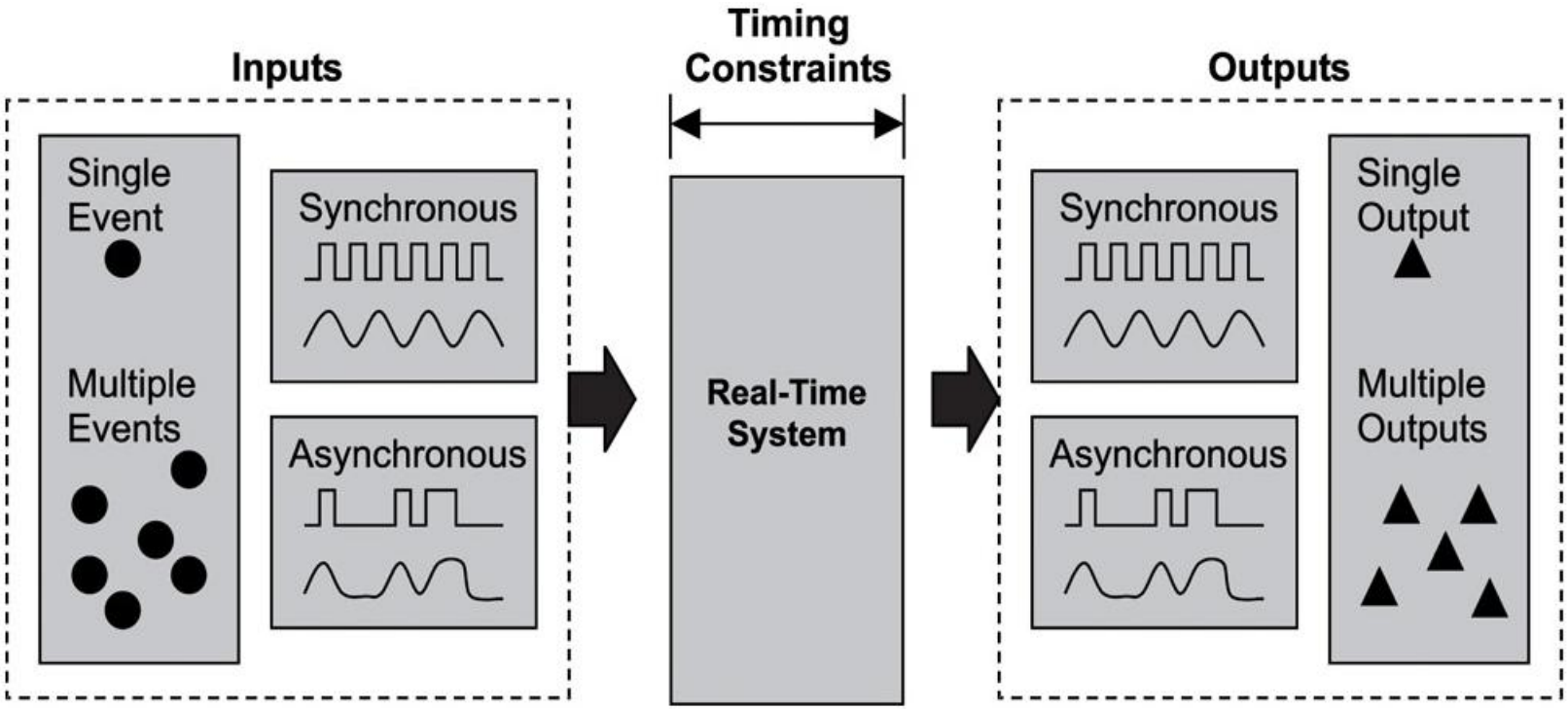
- The overall correctness of a real-time system depends on both the *functional correctness* and the *timing correctness*

Issues & challenges in RTS

https://www.eventhelix.com/embedded/issues-in-real-time-system-design/

Introduction: Real-Time System

- ***Real-time system***: respond to external events in a timely fashion and the response time is guaranteed



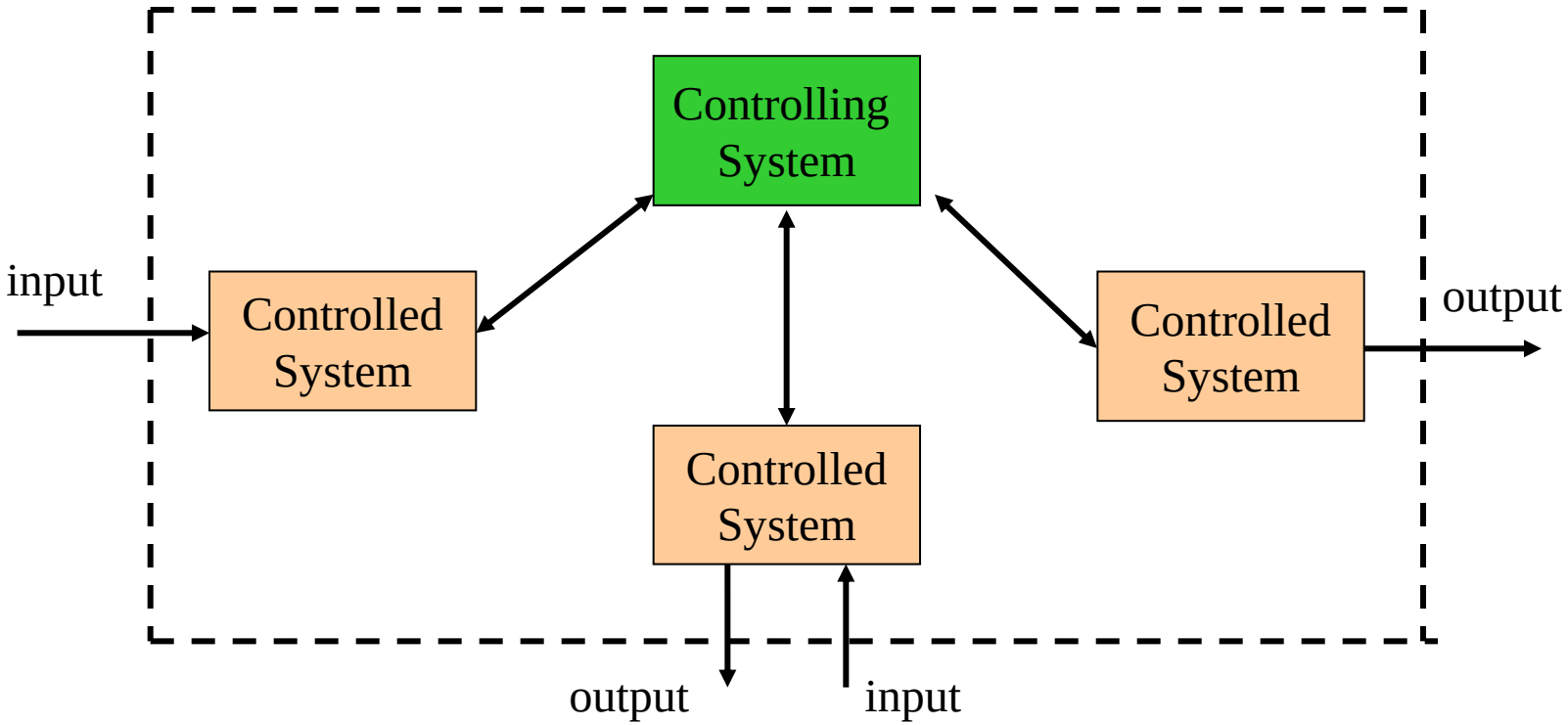
Introduction: Real-Time System

- Responding to external events includes
  - Recognize when an event occurs
  - Perform the required processing
  - Output the result within a given time constraint

- Timing constraints include
  - Finish time
  - Both start time and finish time

Real-Time Systems

- The structure of a real-time system is a *controlling system* and at least one *controlled system*



Real-Time Systems (Cont.)

- The controlling system interacts with the controlled system in various ways
  - The interaction can be *periodic*
    - ***The controlling system -> the controlled system***
    - ***Predictable and occurs at predefined intervals***
  - The interaction can be *aperiodic*
    - ***The controlled system -> the controlling system***
    - ***Unpredictable from random occurrences of external events***
  - The communication can be a combination of both types
- The controlling system must process and respond to the events and information generated by the controlled system in a guaranteed time frame.

Real-Time Systems Examples

- Real-time weapons defense system
  - Controlling system
    - A command-and-decision (C&D) system
  - Controlled system
    - A radar system
    - Weapons firing control system
  - ***Communication between the radar system and C&D system is aperiodic***
  - ***Communication between the C&D system and the weapon firing control system is periodic***

Real-Time Systems Examples (Cont.)

- Cruise missile guidance system
  - Controlling system
    - Navigation system: contain digital maps covering the missile flight path
  - Controlled system
    - Radar system
    - Divert-and-altitude-control system
  - Communication between radars and the navigation system is *aperidoic*
  - Communication between the navigation system and the diver-and-altitude-control system is *periodic*

Real-Time Systems Examples (Cont.)

- DVD player
  - Controlling system
    - DVD player must decode both the video and audio streams from the disc simultaneously.
  - Controlled system
    - Remote control is viewed as a sensor to feed pause and language selection events into DVD player

Type of Real-Time System

- Hard Real-Time
- Soft Real-Time
- Firm

A real-time system means that the system is subjected to real-time, i.e., the response should be guaranteed within a specified timing constraint or the system should meet the specified deadline. For example flight control systems, real-time monitors, etc.

Types of real-time systems based on timing constraints:

1.Hard real-time system: This type of system can never miss its deadline. Missing the deadline may have disastrous consequences. The usefulness of results produced by a hard real-time system decreases abruptly and may become negative if tardiness increases. Tardiness means how late a real-time system completes its task with respect to its deadline. Example: Flight controller system.

2.Soft real-time system: This type of system can miss its deadline occasionally with some acceptably low probability. Missing the deadline have no disastrous consequences. The usefulness of results produced by a soft real-time system decreases gradually with an increase in tardiness. Example: Telephone switches.

3.Firm Real-Time Systems: These are systems that lie between hard and soft real-time systems. In firm real-time systems, missing a deadline is tolerable, but the usefulness of the output decreases with time. Examples of firm real-time systems include online trading systems, online auction systems, and reservation systems.

Hard Real-Time System

- Degree of tolerance of missed deadline
  - Extremely small or zero
- Usefulness of computed results after missed deadlines
  - Likely useless
- Severity of the penalty incurred for failing to meet deadlines
  - catastrophe

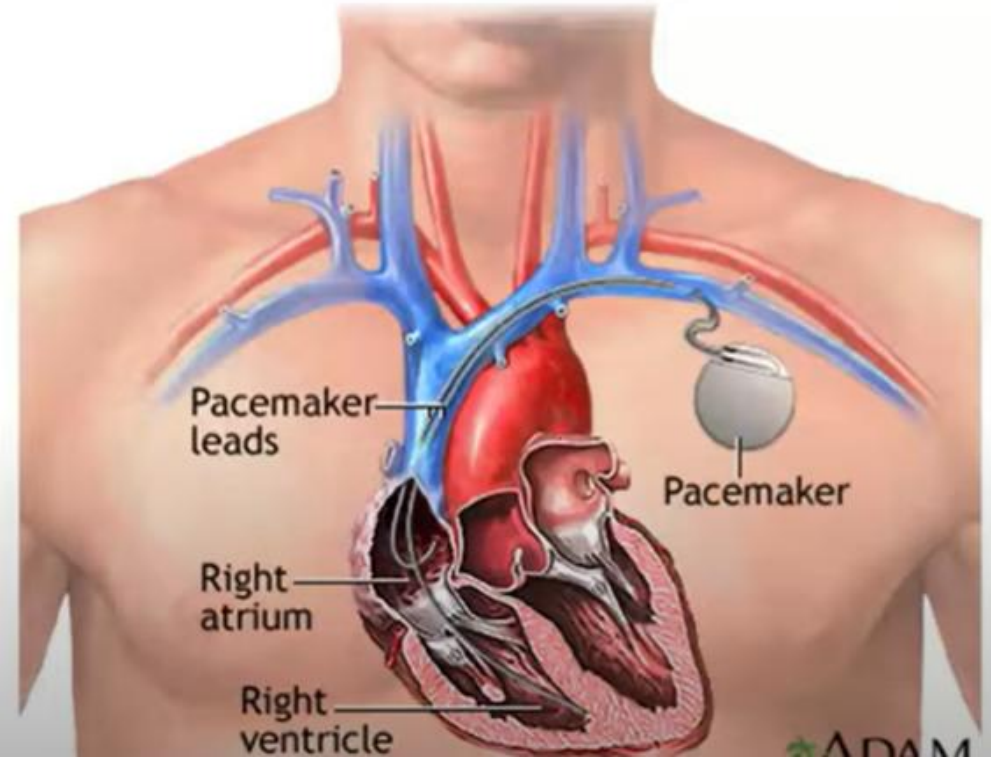
https://www.toppr.com/guides/computer-science/computer-fundamentals/operating-system/real-time-operating-system-rtos/

Hard Real-Time System

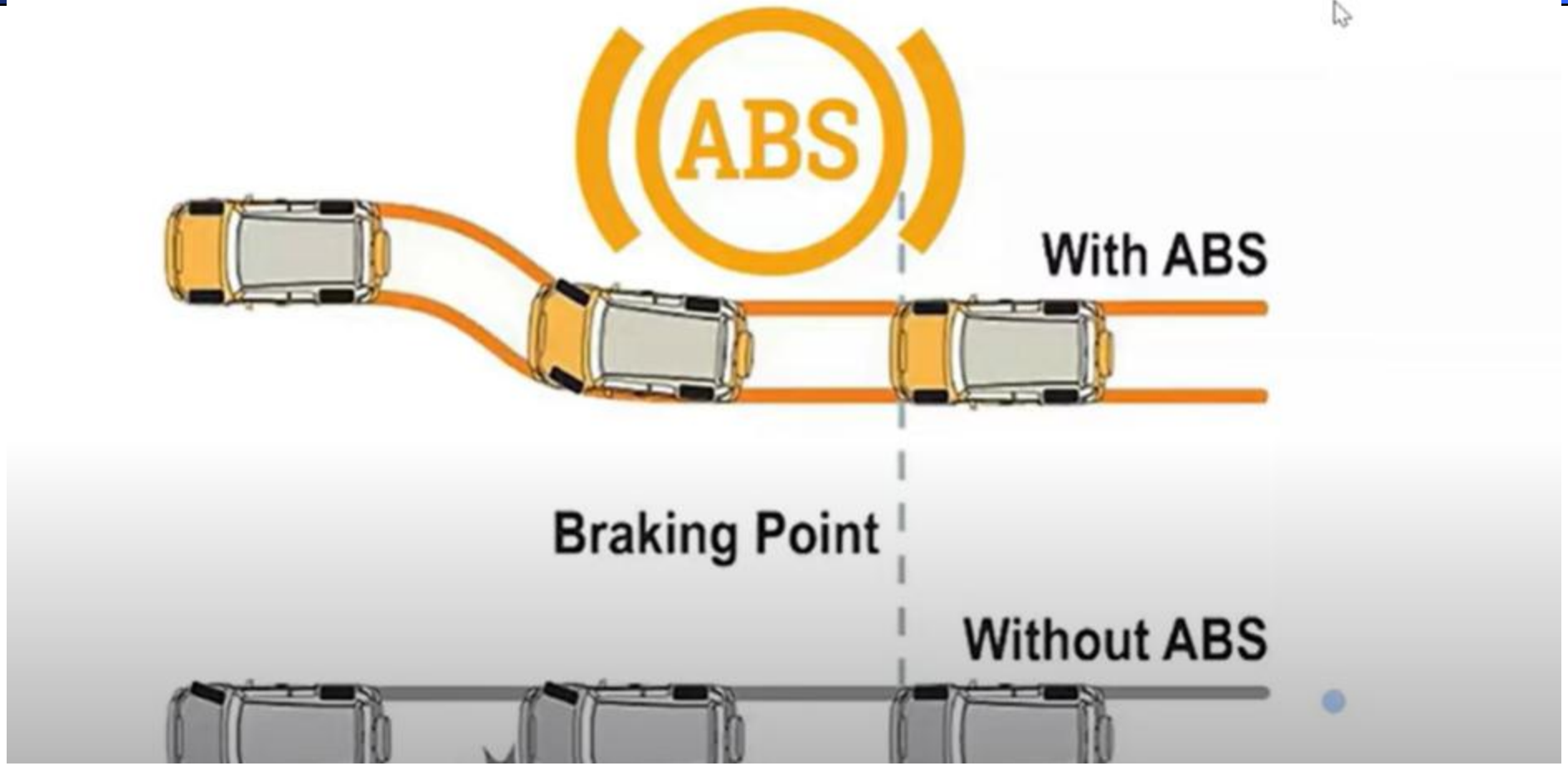
- Examples:
  - Nuclear reactors
  - Flight controller
  - Weapon defense system
  - Missile guidance system

Hard Real-Time System

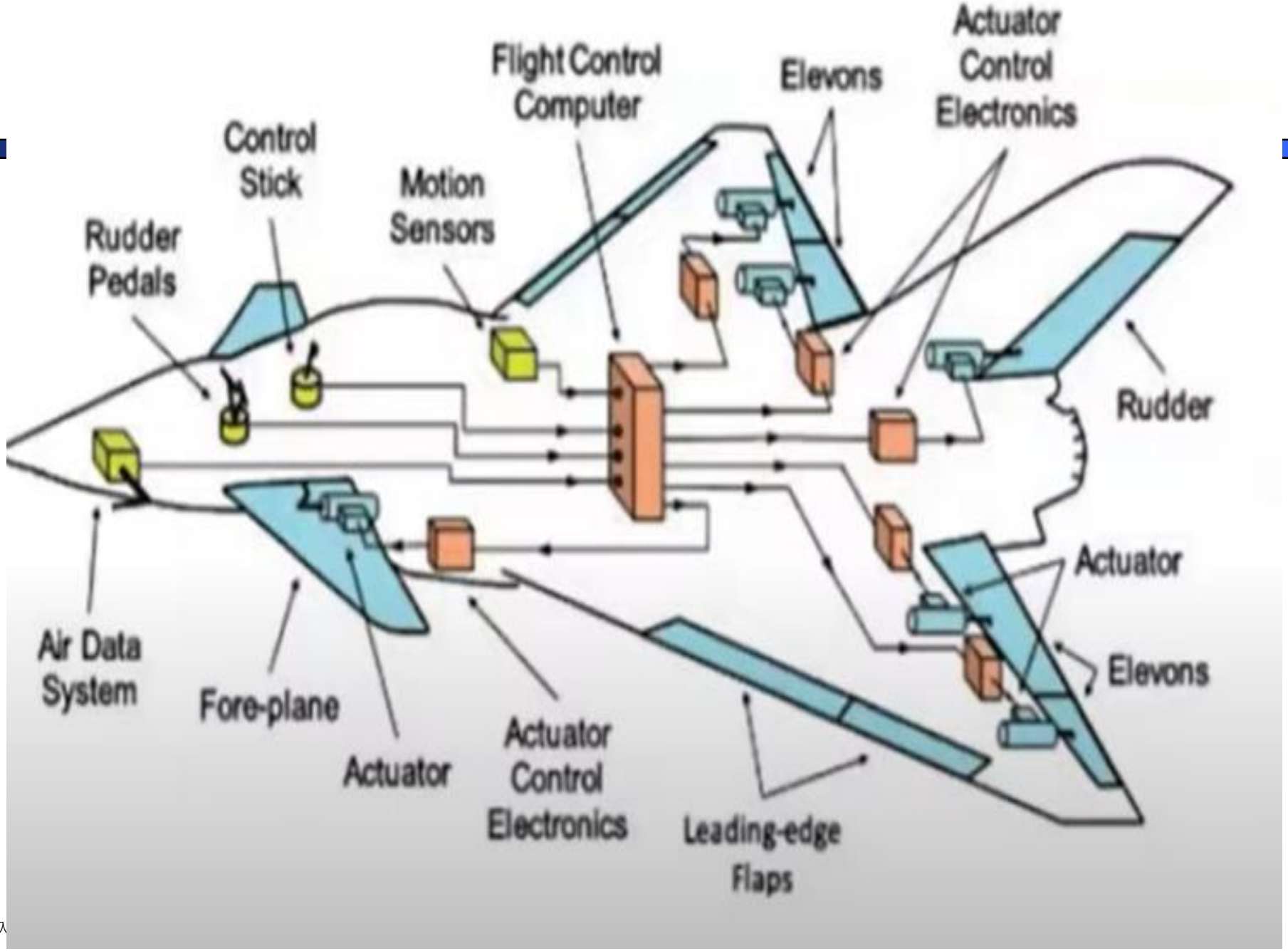
Hard Real-Time System



HArd Real-Time System



Hard Real-Time System


Flight Control Computer
Elevons
Actuator Control Electronics
Control Stick
Motion Sensors
Rudder Pedals
Rudder
Actuator
Elevons
Air Data System
Fore-plane
Actuator
Actuator Control Electronics
Leading-edge Flaps

Soft Real-Time System

- Degree of tolerance of missed deadline
  - Non-zero
- Usefulness of computed results after missed deadlines
  - Have a rate of depreciation
- Severity of the penalty incurred for failing to meet deadlines
  - Non-catastrophic

Soft Real-Time System

- A miss of timing constraints is undesirable. However, a few misses do no serious harm
  - The timing requirements are often specified in probability terms
  - The time constraints are guaranteed on a statistical basis

- Examples:
  - Multimedia Streaming
  - Electronic games
  - Quality-of-Service (QoS) guarantees

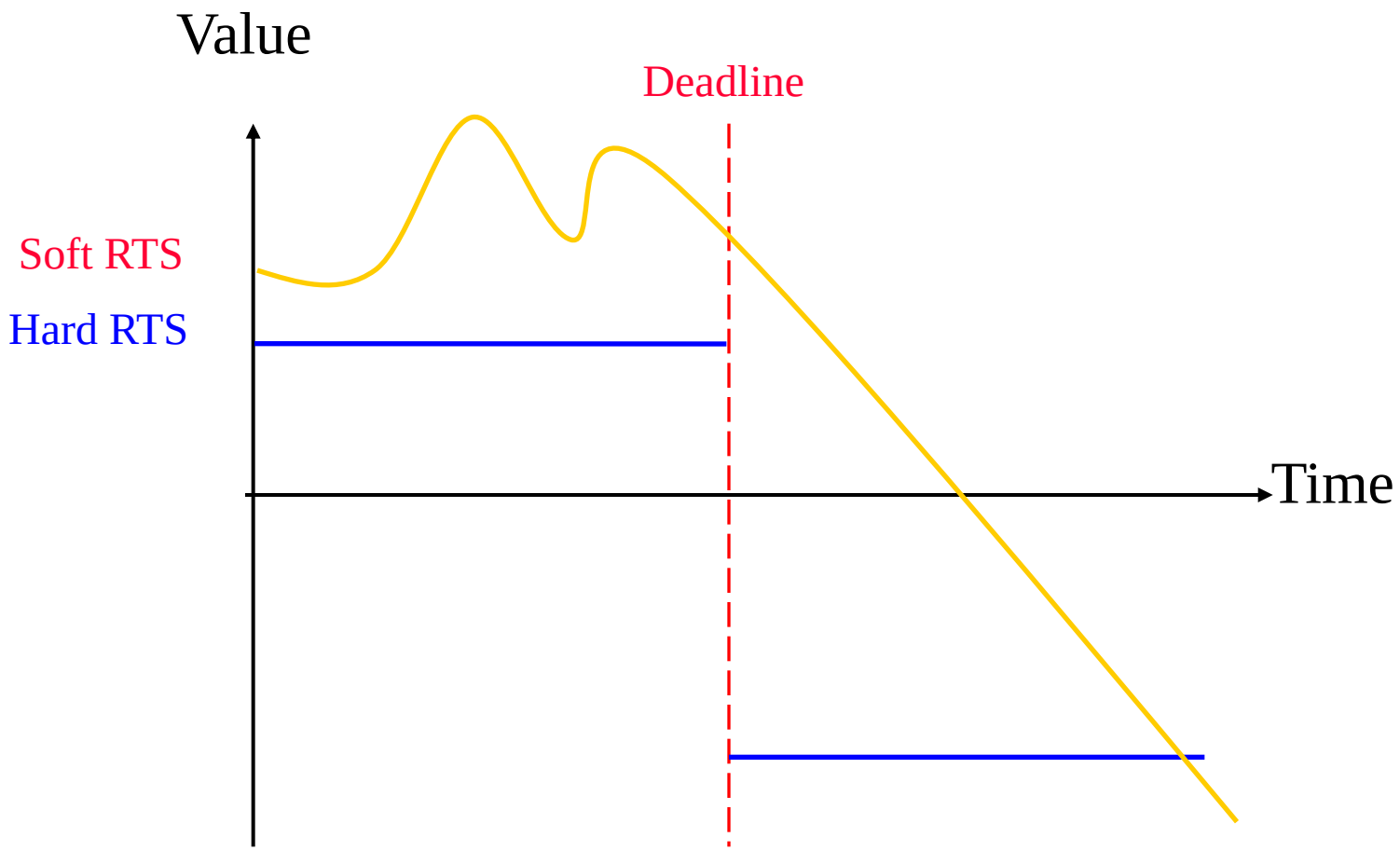
Soft Real-Time System

Soft Real-Time System

Soft Real-Time System

https://testbook.com/key-differences/difference-between-hard-real-time-and-soft-real-time-system#:~:text=Examples%20of%20hard%20real%2Dtime,multimedia%20systems%2C%20and%20email%20systems.

Comparison



Job versus Task

- Job: unit of work that is scheduled and executed by the system
  - Computation of a FFT (Fast Fourier Transform)
  - Transmission of a data packet

- Task: a set of related jobs which jointly provide some system function

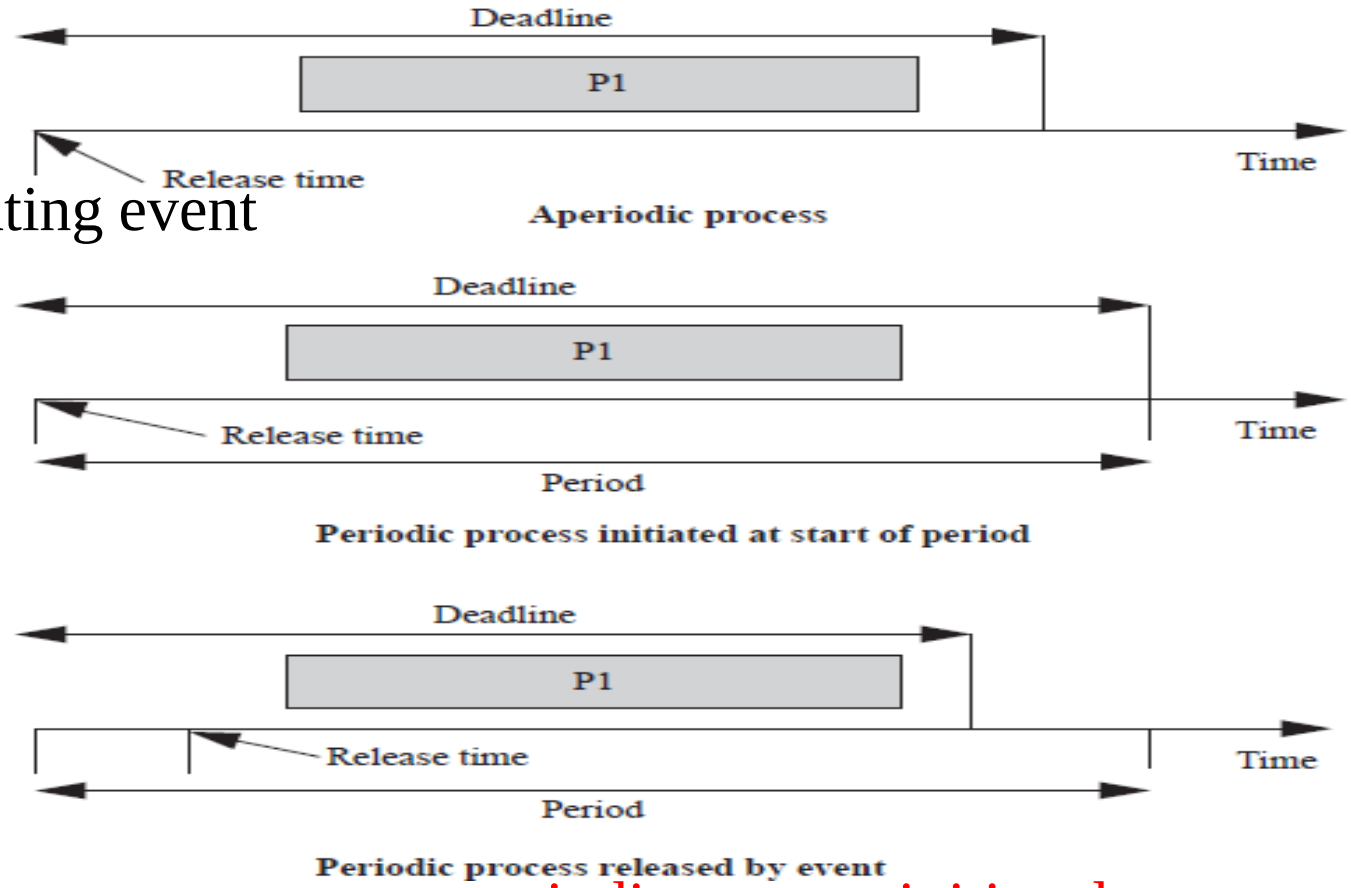
Release Time and Deadlines

- Release time
  - the instant of time at which the job becomes available for execution
  - Jobs have no release time if all the jobs are released when the system begins execution

- Response time
  - The length of time from the release time of the job to the instant when it completes

Timing specifications on processes

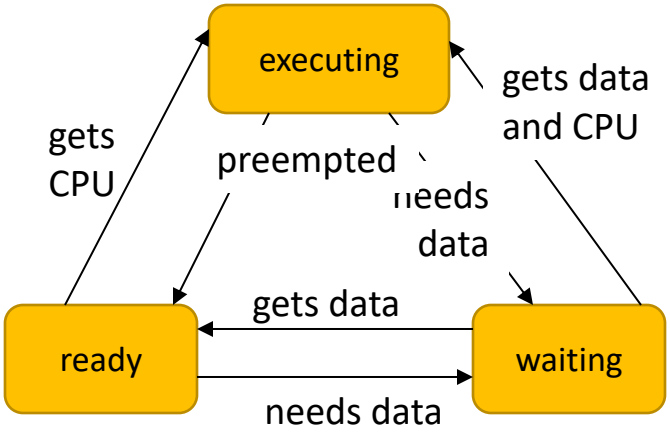
- Release time: time at which process becomes ready.
- Deadline: time at which process must finish.



periodic process initiated by event

State of a process

- A process can be in one of three states:
  - executing on the CPU;
  - ready to run;
  - waiting for data.



Utilization

- CPU utilization:
  - Fraction of the CPU that is doing useful work.
  - Often calculated assuming no scheduling overhead.
- Utilization:
  - U = (CPU time for useful work)/ (total available CPU time)

$$= \frac{\sum_{t_1}^{t_2} T(t)}{t_2 - t_1}$$

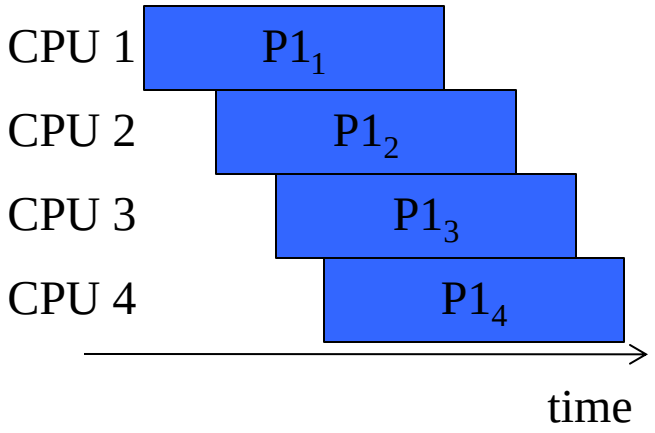
$$= T/t$$

The scheduling problem

- Can we meet all deadlines?
  - Must be able to meet deadlines in all cases.
- How much CPU horsepower do we need to meet our deadlines?

Rate requirements on processes

- Period: interval between process activations.
- Rate: reciprocal of period.
- Initiation rate may be higher than period--- several copies of process run at once.



Release Time and Deadlines (Cont.)

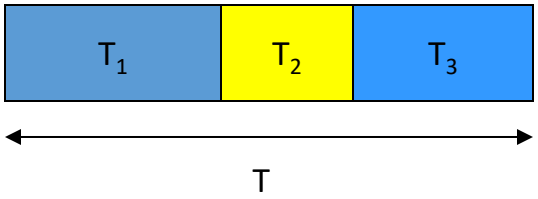
- Relative deadline
  - The maximum allowable response time of a job
- Deadline or Absolute Deadline
  - The instant of time by which its execution is required to be completed
  - Equal to the release time plus the relative deadline
  - A job has no deadline if its deadline is at infinity

Feasibility

- Timing Constraints
  - A constraints imposed on the timing behavior of a job
  - Specified in terms of
    - Release time
    - Relative deadline or absolute deadline
- The set of rules that determines the order in which tasks are executed is called a *scheduling algorithm*
  - A schedule is *feasible* if all tasks can be completed according to the timing constraints- Must show that the deadlines are met for all timings of resource requests
  - A set of tasks is *schedulable* if there exists at least one algorithm that can produce a feasible schedule

Simple processor feasibility

- Assume:
  - No resource conflicts.
  - Constant process execution times.
- Require:
  - $T \geq \Sigma_i T_i$
  - Can't use more than 100% of the CPU.



Events or signals could be classified as

a. Arrival

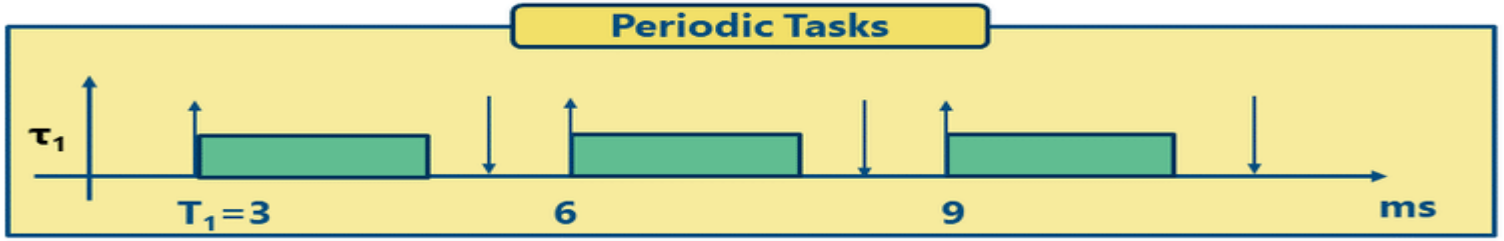
b. Form or Structure

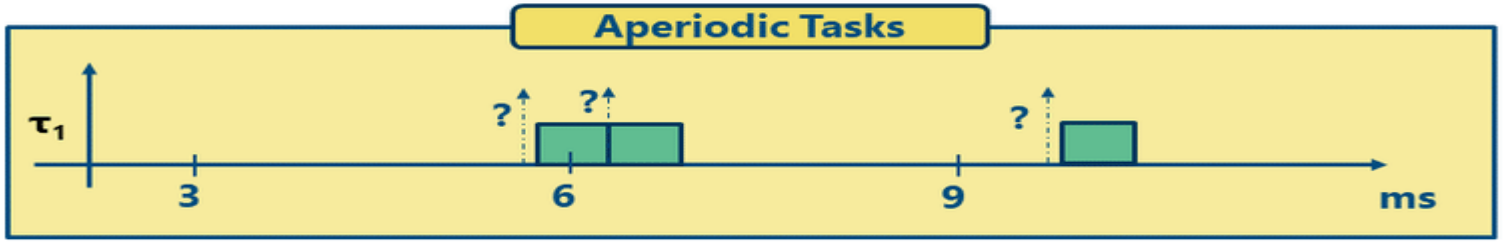
c. Type (hint: what it conveys?)

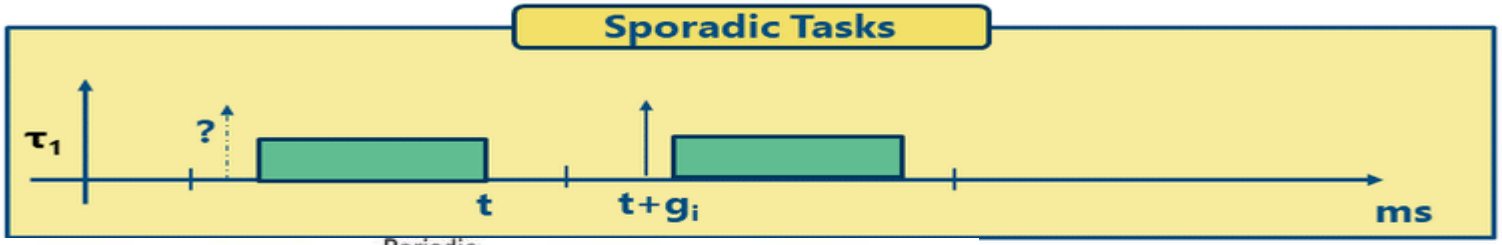
Type of Tasks

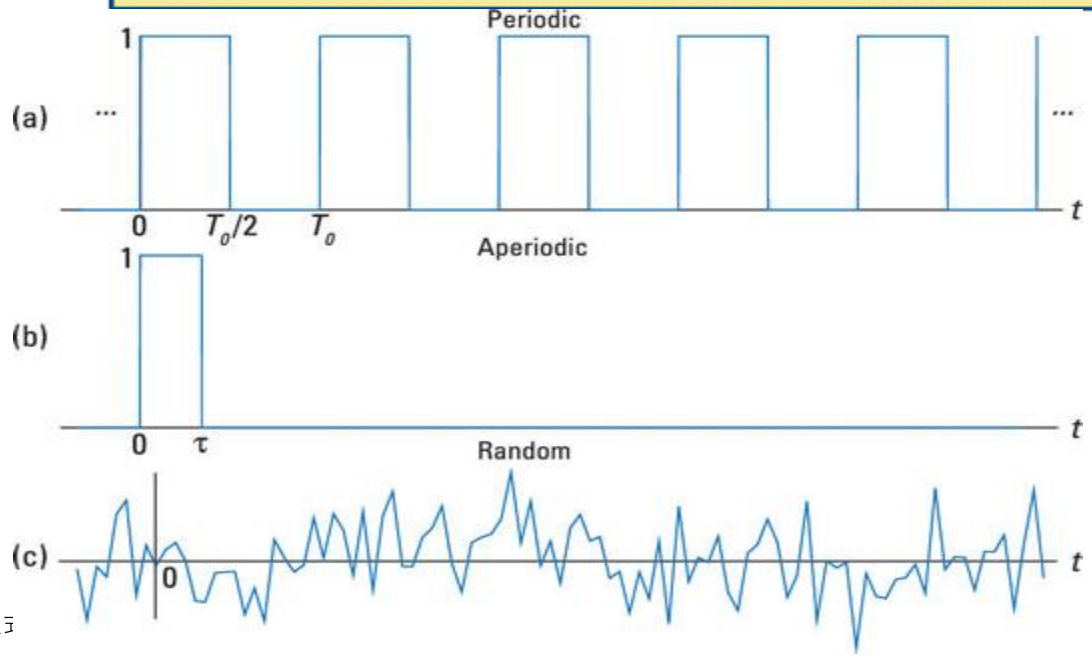
- *A **Periodic task*** is executed repeatly at regular time intervals, and each invocation is called a *job* or *instance*.
  - Often *time-driven*
- *A **Aperiodic task*** is executed to response to external events, and to respond, it executes aperiodic jobs whose release time are not know a priori -Often *event-driven*
  - Off-line guarantee of aperiodic tasks must make proper assumptions on the environments; that is, by assuming a maximum arrival rate for each event (i.e., minimum interarrival time)
  - Aperiodic tasks characterized by a minimum interarrival time are called ***sporadic tasks (do not have hard deadlines.)***

An aperiodic signal is a signal that doesn't repeat itself after a specific time interval, and can't be represented by a single fundamental period. Aperiodic signals are also known as non-periodic signals.









Aperiodic signals or non periodic are signals that don't have a period, or an irregular pattern of repetition. However, some aperiodic signals are predictable, while others are random

These signals are non-periodic, meaning they arise from unpredictable disturbances

The scheduling algorithms can be classified as

1.Preemptive or non-preemptive

2.Static or dynamic

3.Off-line or on-line

4.Optimal or heuristic

5.Dynamic or fixed

Classification of Scheduling Algorithm

- Preemptive/Nonpreemptive
  - **Preemptive:** the running tasks can be interrupted to assign the processor to another task
  - Non-preemptive: A task, once started, is executed until completion.
- Static/Dynamic
  - Static: the scheduling decisions are based on fixed parameters and assigned to tasks before their activation
  - Dynamic: the scheduling decisions are based on dynamic parameters that may change during system evolution

Classification of Scheduling Algorithm (Cont.)

- Off-line/On-line
  - Off-line: a scheduling algorithm is executed on the entire task set before actual task activation. The schedule may be stored in a table and later executed by a dispatcher.
  - On-line: scheduling decisions are taken at runtime every time a new task enter the system or when a running task terminates

Classification of Scheduling Algorithm (Cont.)

- Optimal/Heuristic
  - Optimal: an algorithm is optimal if it minimizes some given cost function defined over the task set.
    - If no cost function is given and only concern feasibility, an algorithm is optimal if it may fail to meet deadline only if no other algorithms can meet it
  - Heuristic: an algorithm is heuristic if it tends toward but does not guarantee to find the optimal schedule
- Dynamic/Fixed
  - Fixed: in which the priority of each process is fixed for any instantiation
  - Dynamic: in contrast from above

Metrics for Performance Evaluation

- Metrics for hard real-time system
  - Schedulability
  - Optimality, etc.
- Metrics for soft real-time system
  - Miss ratio
  - Response time, etc.
- Other metrics
  - Completion time
  - Jitter, etc.
  - Combination of metrics

Commonly Used Approaches to Real-Time Scheduling

- Clock-Driven
- Priority-Driven

Clock-Driven Approach

- Also called time-driven
  - Because each scheduling decision is make at a specific time, independent of events, such as job releases or completions, in the system
- Applicable only when the system is deterministic, except for a few aperiodic jobs
- The parameters of all periodic tasks are known a priori
- Therefore, the scheduler is often constructed by a *static schedule* of the jobs off-line
  - Use a hardware timer to trigger the scheduling decision

Priority-Driven Approach

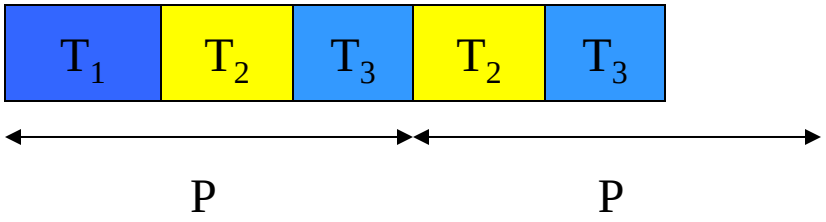
- Scheduling decisions are made when events, such as release and completions of job occurs
  - Therefore, priority-driven algorithms are *even-driven*
- Many non-real-time scheduling algorithms are priority-driven
  - FIFO (First-In-First-Out)
    - Assign priority according to the release time
  - SETF (Shortest-Execution-Time-First)
    - Assign priority on the basis of job execution times

Priority-Driven Approach (Cont.)

- Priority-driven real-time scheduling algorithms
  - Dynamic-priority
    - e.g. EDF (Earliest Deadline First)
  - Fixed-priority
    - e.g. RM (Rate Monotonic)

Round-robin

- Schedule process only if ready.
  - Always test processes in the same order.
- Variations:
  - Constant system period.
  - Start round-robin again after finishing a round.



Round-robin assumptions

- Schedule based on least common multiple (LCM) of the process periods.
- Best done with equal time slots for processes.
- Simple scheduler -> low scheduling overhead.
  - Can be implemented in hardware.

Round-robin schedulability

- Can bound maximum CPU load.
  - May leave unused CPU cycles.
- Can be adapted to handle unexpected load.
  - Use time slots at end of period.